

A Reconfigurable FTL Architecture for NAND Flash-Based Applications

CHANIK PARK et al., Samsung Electronics & KAIST
ACM TECS Vol 7, No 4, Article 38, July 2008

2025. 10. 15

Presenter. 202126852 Ha Janghan

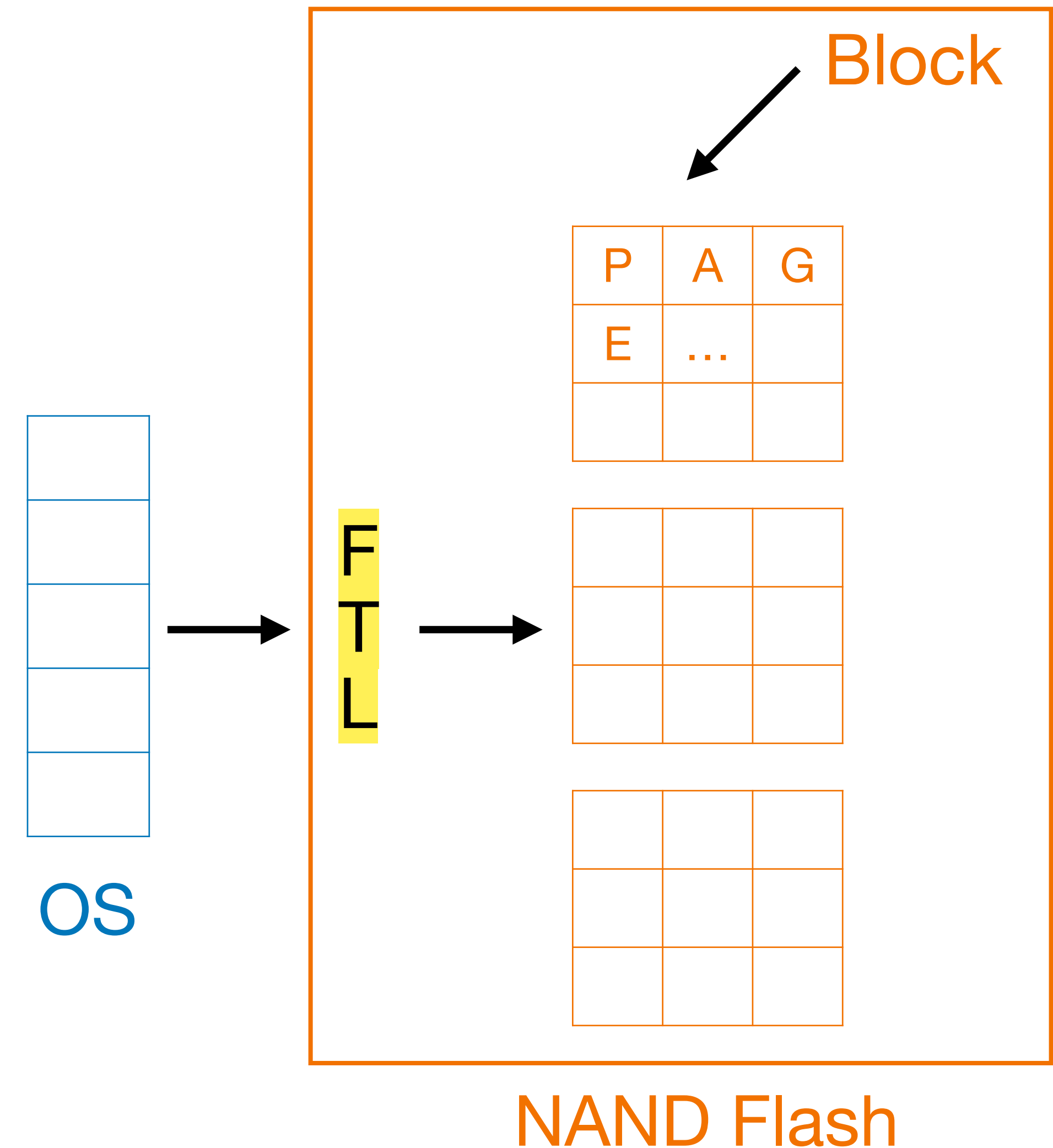
Table of Contents

Prerequisites	Introduction	Background	Mapping Schemes
Proposal	Model	Findings	Conclusion

Prerequisites :

What is FTL?

- OS : 스토리지를 **선형적인 주소 공간**으로 인식.
- NAND Flash : **페이지 / 블록** 단위로 구성.
- **FTL(Flash Translation Layer)** :
논리적 주소와 **물리적 주소**를 매핑하는 계층.



Prerequisites :

NAND Flash Characteristics

- NAND Flash :
 - Write(200us) : 1 → 0 가능
 - Overwrite : 0 → 1 불가능
 - Erase(1.5ms) : 블록 단위로 초기화 필요(erase-before-write)
- erase-before-write 多 → 수명저하, 오버헤드

Introduction

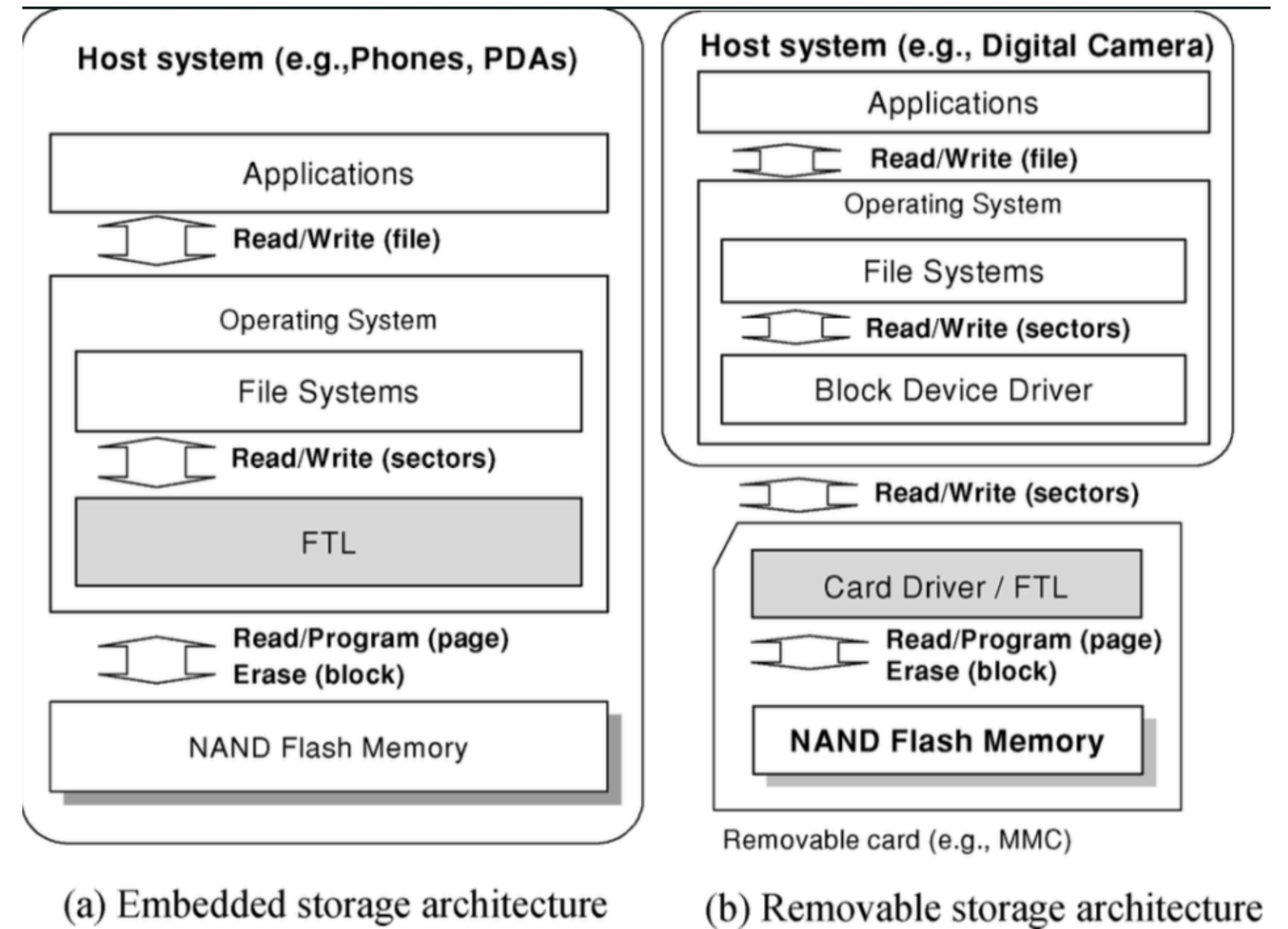
#1

- NAND Flash
 - 비휘발성
 - 저전력
 - 신뢰성
 - 충격내성
- 모바일 시장 점유율 ↑

Introduction

#2

- 하드웨어적 특성상 시스템별 최적화 필요
- 공간적 / 시간적 지역성을 매개변수화한 유연한 FTL 아키텍처 제안.



Background :

NAND Flash Architecture

- 명령코드 + LPN $\rightarrow$ PPN Read(20us)
(Logical Page Number) $\rightarrow$ (Physical Page Number)
- // + 데이터 $\rightarrow$ PPN Program(200us)
- 명령코드 + LBN $\rightarrow$ PBN Erase(1.5ms)
(Logical Block Number) $\rightarrow$ (Physical Block Number)
- 최적화 $\rightarrow$ Write / Erase의 최소화.

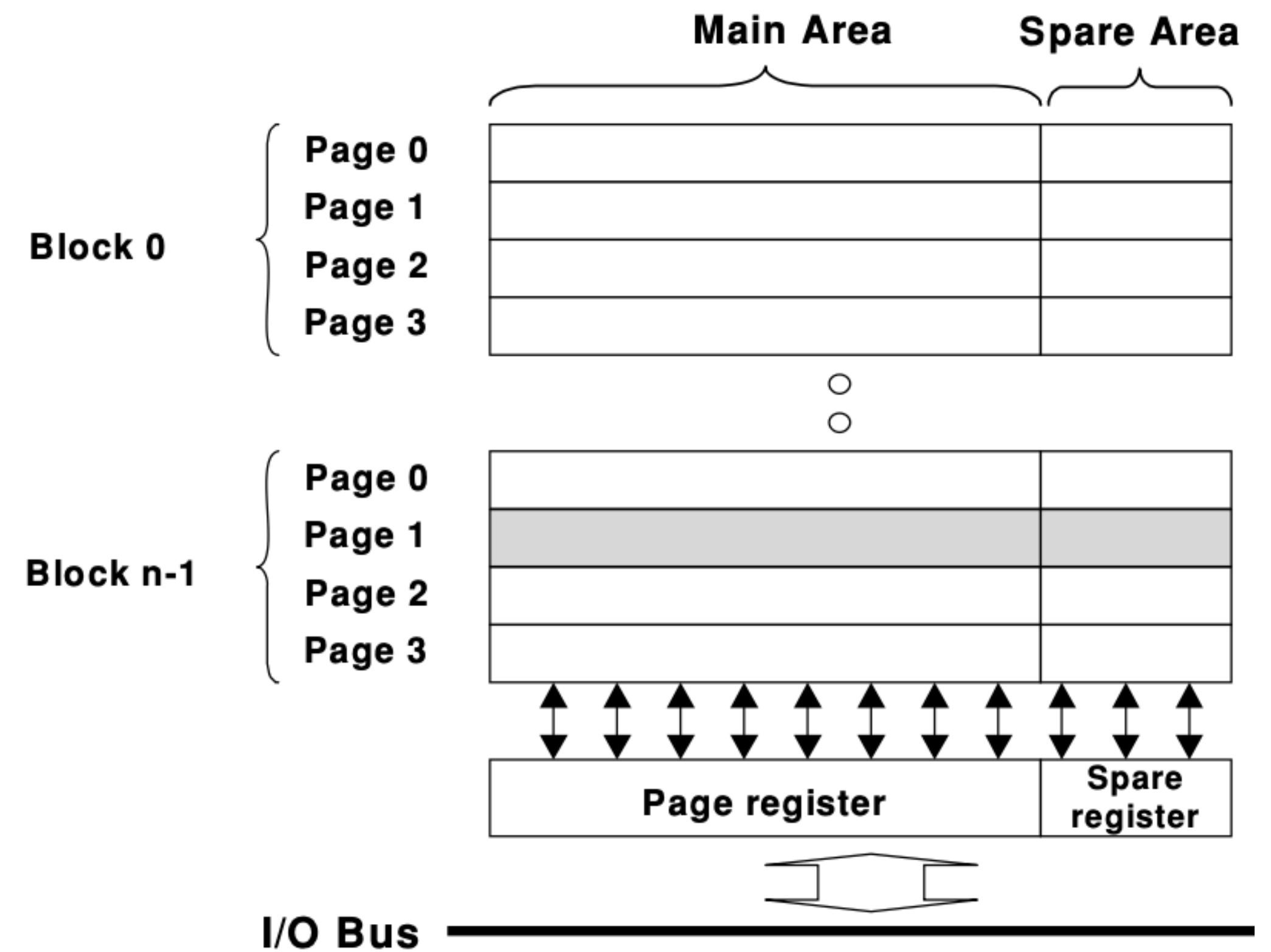


Fig. 3. NAND flash structure.

Background :

FTL Concepts

- 전통적인 FTL → NAND Flash를 메타데이터 / 유저 데이터 블록으로 분할.
- 메타데이터 블록 →
 - Map Block → 논리적 주소를 물리적 주소로 변환하는 블록.
 - Write Buffer Block → 쓸 데이터를 위한 버퍼 블록.
 - Reserved Block → 배드 블록을 대체하기 위한 여분 블록.
- 유저 데이터 블록 → 고정된 수의 데이터 블록으로 구성.

Basic Mapping Schemes :

Page Mapping

- NAND Flash, 호스트의 메모리 양쪽에서 매핑 테이블을 관리.
- 발견한 페이지가 유효할 경우, 다른 페이지에 Write.

✓ erase-before-write 연산 ↓↓.

✓ 데이터의 유연한 관리.

✗ Garbage Collection 필수.

✗ 매핑 테이블 크기 ↑.

Basic Mapping Schemes :

Block Mapping

- $LPN \rightarrow LBN + \text{페이지 오프셋}$

(Logical Block Number)

- $PBN \rightarrow PBN + \text{페이지 오프셋}$

(Physical Block Number)

✓ 매핑 테이블 크기 ↓ ↓. (1MB → 16KB)

✓ 순차적 쓰기 성능 ↑.

✗ erase-before-write 연산 ↑.

✗ 무작위 쓰기 성능 ↓ ↓.

Hybrid Mapping Schemes :

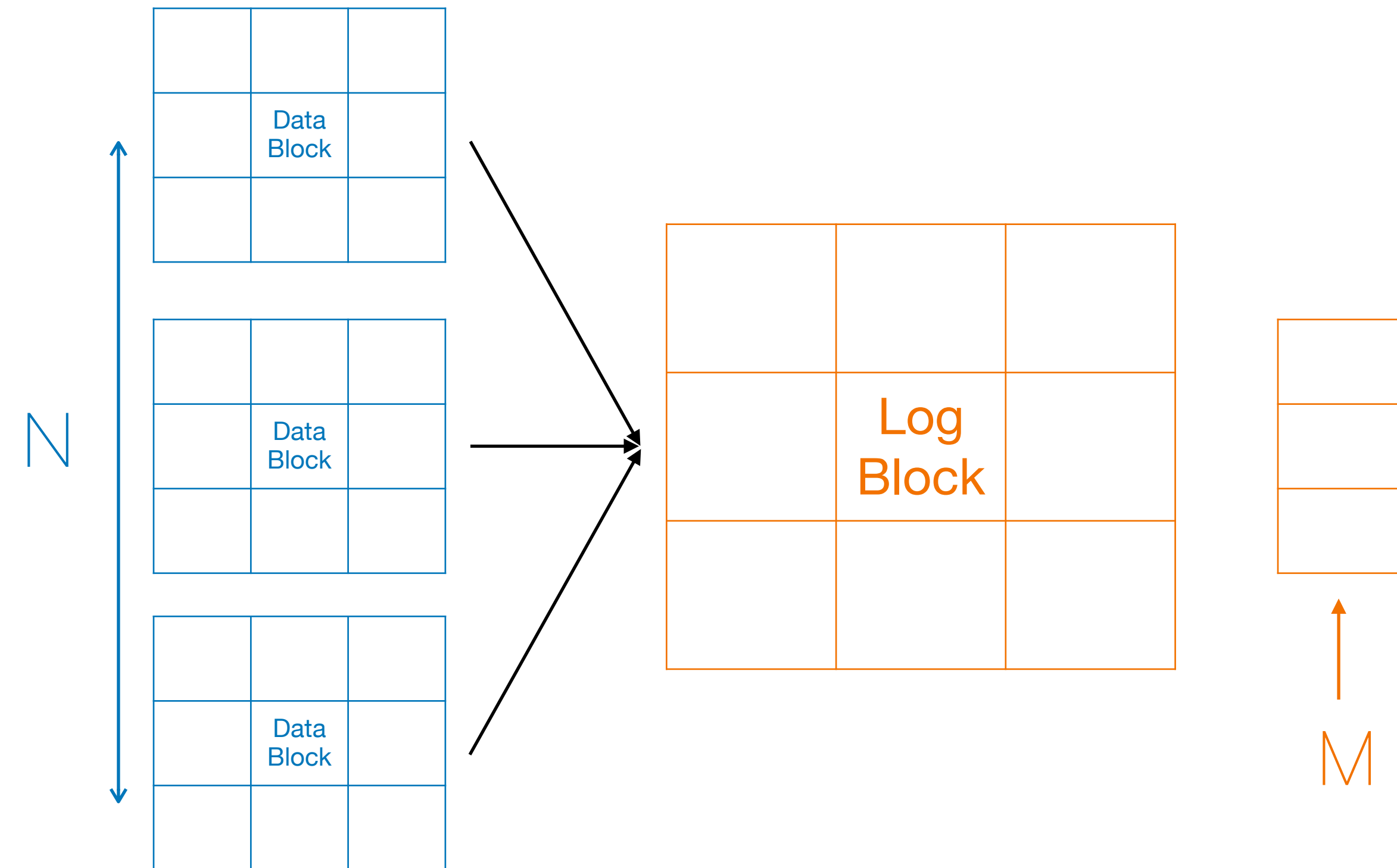
N to N + M Mapping

- N개의 데이터 블록 → N + M개의 로그 블록
 - N → 데이터 블록 그룹의 크기
 - 근처까지 참조하는 경우 → 공간 지역성
 - M → 런타임 추가 로그 블록 수
 - 반복적으로 참조하는 경우 → 시간 지역성

✓ GC 오버헤드 ↓↓

✓ Merge 효율 ↑

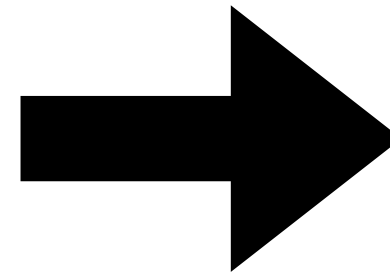
✗ 어플리케이션에 따른 최적화 난이도 X



Proposal :

Flexible Group Mapping + Methodology

- N to N + M Mapping
 - $N \leftarrow$ 다양한 Workload 반영 X
 - $M \leftarrow$ 통제불가능



- Flexible Group Mapping
 - $N \leftarrow$ 어플리케이션에 따라 조정
 - $K \leftarrow M$ 의 최대값 설정
- + N, K 탐색 방법론

Implementation :

Write Scheme

Map Table	Index	Value	Table Type
DBMT (Data Block Mapping Table)	LBN (Logical Block Number)	PBN (Physical Block Number)	Array
LBMT (Log Block Mapping Table)	DGN (Data Group Number)	PBN	Set
LPMT (Log Page Mapping Table)	DGN	(LPN, PPN) (Logical/Physical Page Number)	Set

Implementation :

Merge Scheme

Base	Scheme	Condition	Result	Copy O / X
Copy	Simple-Merge	기본	$\text{New} \leftarrow \text{로그} + \text{데이터}$ $\text{로그} / \text{데이터} \rightarrow \text{Free}$	O
	Copy-Merge	로그 블록이 최신 페이지로 일부 채워져 있는 경우	$\text{로그} \leftarrow \text{로그} + \text{데이터}$ $\text{데이터} \rightarrow \text{Free}$	O
Address	Swap-Merge	로그 블록이 최신 페이지로 완전히 채워져 있는 경우	$\text{로그} \leftarrow \text{데이터}$ $\text{데이터} \rightarrow \text{Free}$	X

Model :

Workload Analysis

- Write Request Sequence $R = \langle R_0, R_1, \dots, R_{M-1} \rangle$
- Non-Overlapping Write Request Window W_j ,
- Length of $W_j \rightarrow |W|$
- Request Density = $\frac{C_{i,j}}{|W|}$ A. $C_{i,j} \rightarrow W_j$ 의 Request 중 LBN i에 대한 Request 수

$$\sum_{i \in \text{all LBNs}} RD_{i,j} = \frac{1}{|W|} \sum_{i \in \text{all LBNs}} C_{i,j} = 1 \text{ for any requests window } W_j$$

Model :

Associativity Parameter N

- $N \rightarrow$ 데이터 블록 그룹의 크기 / 공간 지역성
- Request Density $\uparrow / \downarrow \rightarrow$ 공간 지역성 $\uparrow / \downarrow \rightarrow N \downarrow / \uparrow$

$$N_j = \lfloor \min(\frac{1}{RD_{i,j}}) \rfloor \text{ for } i \in \text{all LBNS, } W_j \text{ 안에서 } N \text{의 후보값.}$$

- $N \uparrow \rightarrow$ 로그 블록 활용도 $\uparrow$
 $\rightarrow$ Merge 비용 * N
- N_j 의 히스토그램 $\rightarrow$ 최상위 값만을 후보로 평가

Model :

Max Addable Logs Parameter K #1

$$K_{i,0} = 0, K_{i,j} = K_{i,j-1} + d_{i,j} \text{ for } j > 0$$

* If Overwrite 발생 $d_{i,j} = 1$ Else $d_{i,j} = 0$

Model :

Max Addable Logs Parameter K #2

- A. $C_{i,j} \rightarrow W_j$ 의 Request 중 LBN i에 대한 Request 수
- B. $PAGE_{i,j} \rightarrow W_j$ 동안 Write된 LBN i의 Page 집합

- $K \rightarrow$ 로그 블록 그룹에 추가될 수 있는 최대 로그 블록 수 / 시간 지역성

$$d_{i,j} = \begin{cases} 1 & \text{In-Window Overwrite} \\ 0 & \text{otherwise} \end{cases} \quad \text{Window k - Window j Overwrite}$$

$\text{if } |PAGE_{i,j}| < C_{i,j} \vee \bigcup_{k=0..j-1} PAGE_{i,k} \cap PAGE_{i,j} \neq \emptyset$

- $K \uparrow \rightarrow$ Delayed Merge 극대화

$\rightarrow$ 로그 블록이 없는 데이터 블록 간 경쟁 심화 $\rightarrow$ Merge 비용 $\uparrow$

Findings :

Usecase #1

- MS Office + Web Surfing
- Random Access Pattern
- 강한 시간성으로 기존의 파일 업데이트
- 강한 공간성으로 독립적으로 접근

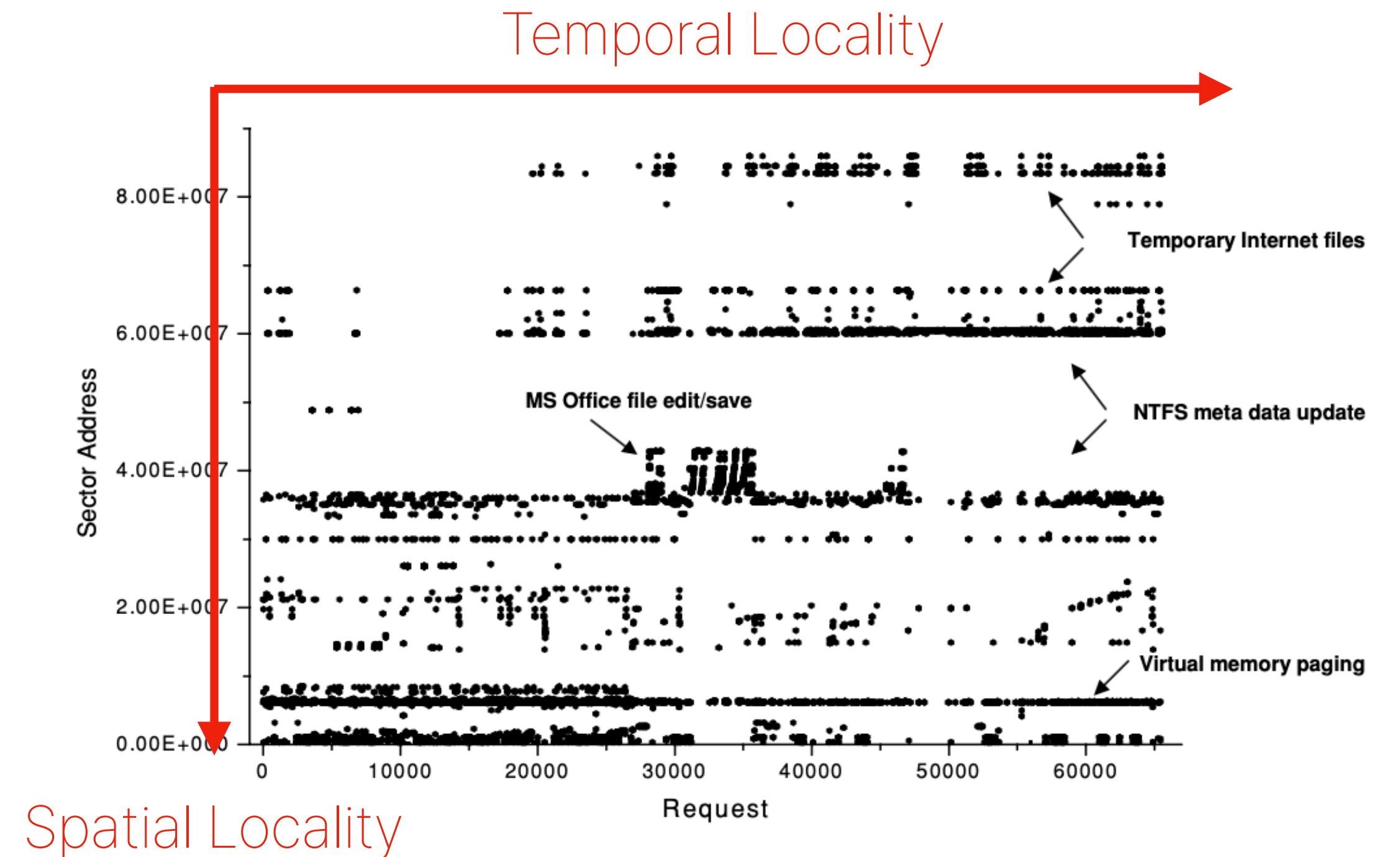


Fig. 14. Trace distribution from PC applications.

Findings :

Results #1

$$N = 2, 4, 8$$

$$K = 4, 8$$

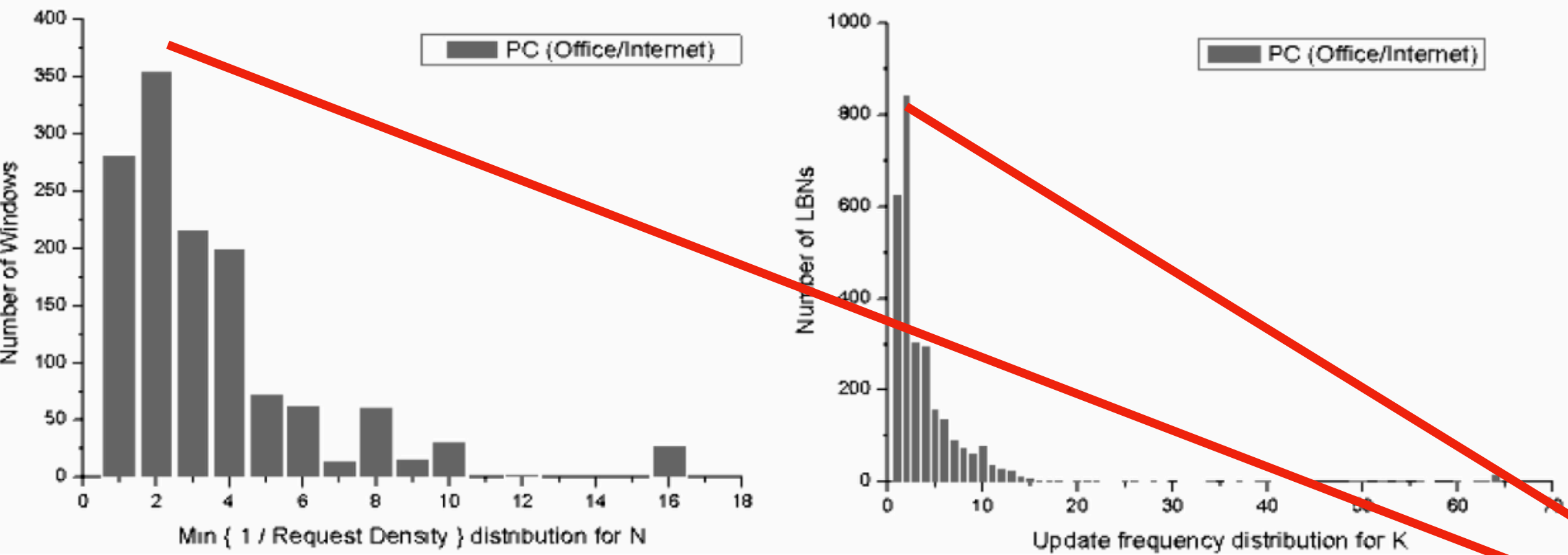
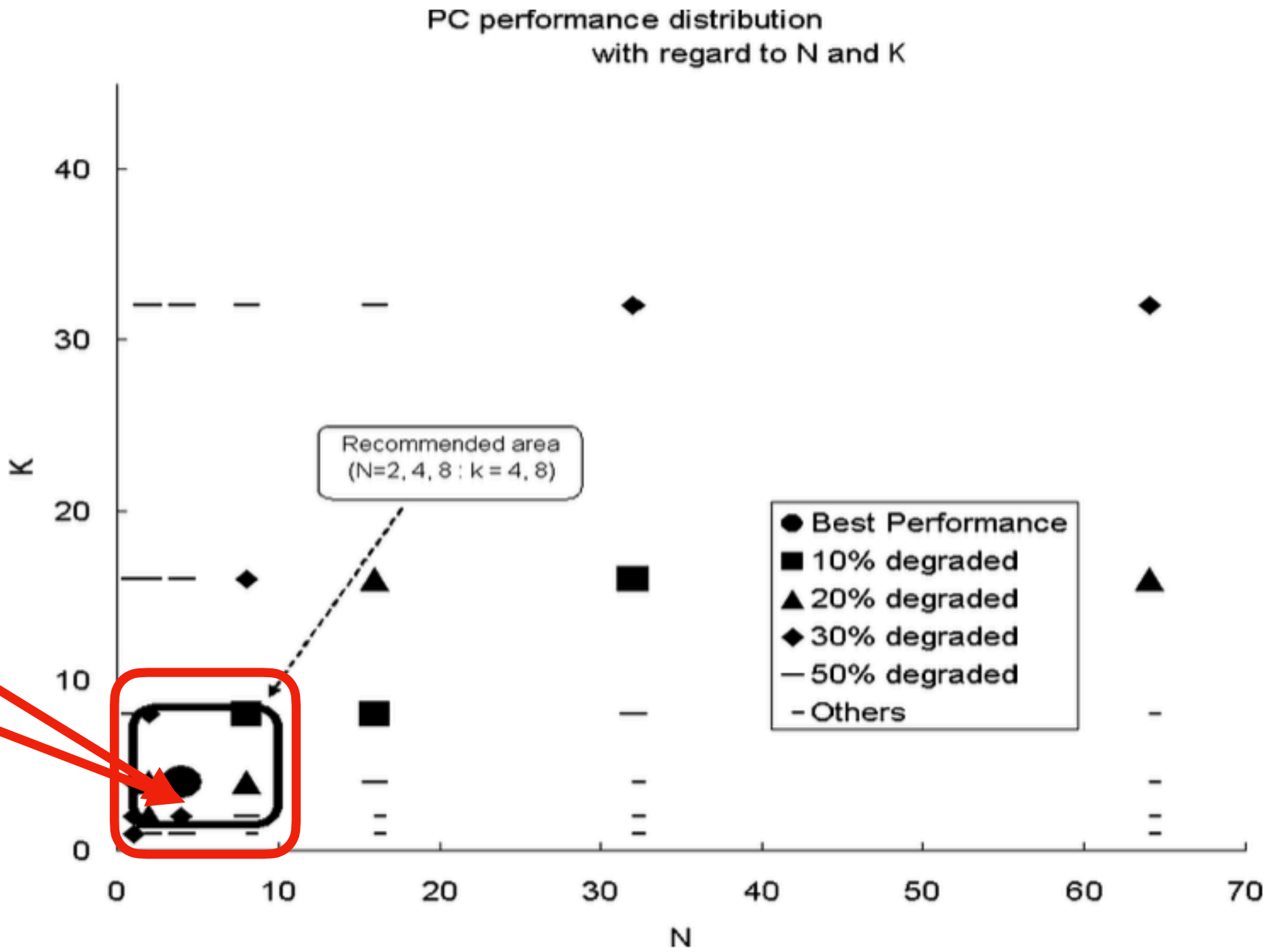


Fig. 16. Distribution of N's and K's for PC applications.



PC application performance variation with the change of N and K .

Findings :

Usecase #2

- MP3 Download + 메타데이터 업데이트
- Sequential Access Pattern
- 강한 공간성으로 순차적으로 접근

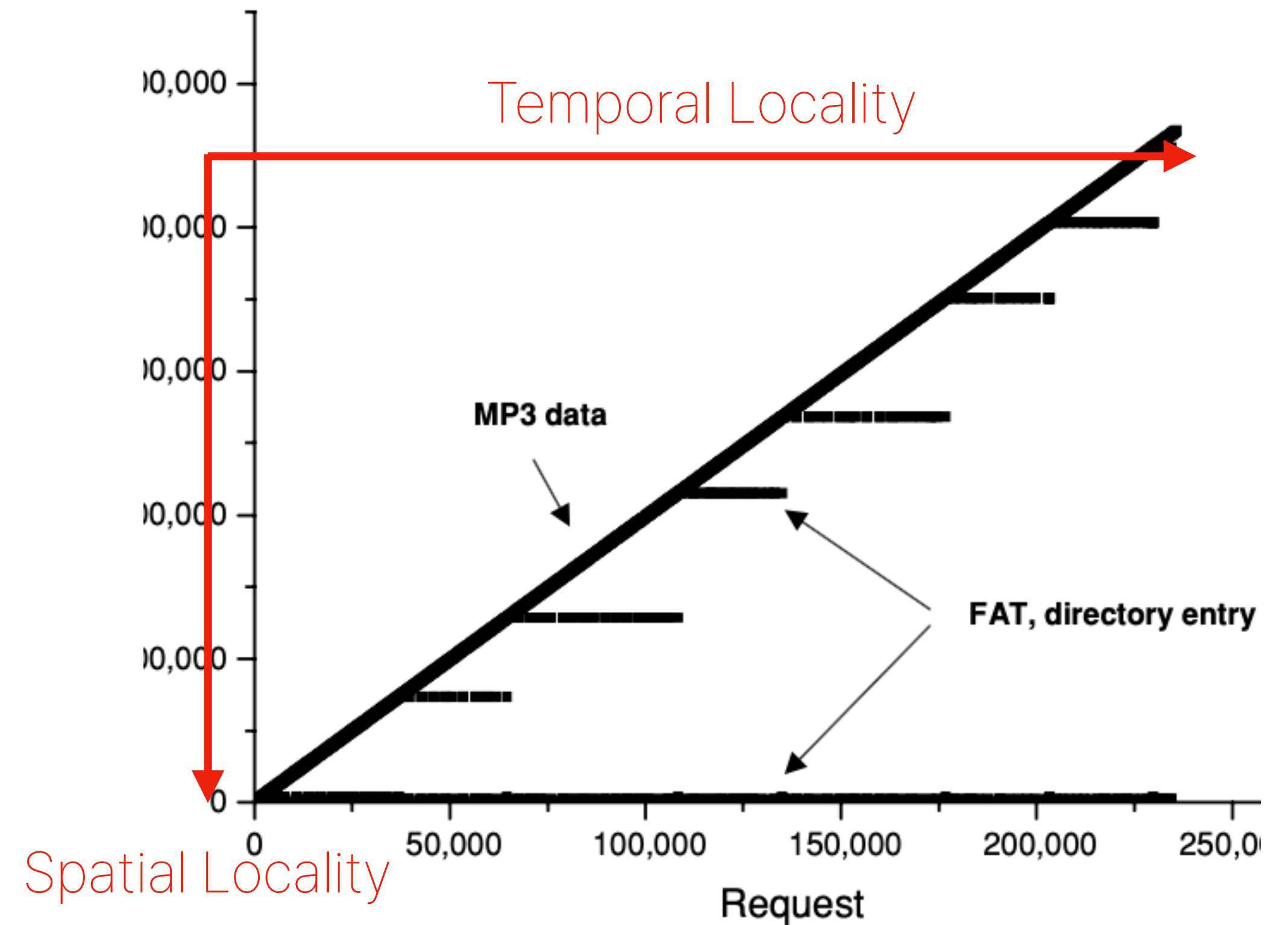


Fig. 15. Trace distribution from an MP3 download.

Findings :

Results #2

$N = 1, 2$

$K = 8 \sim 32$

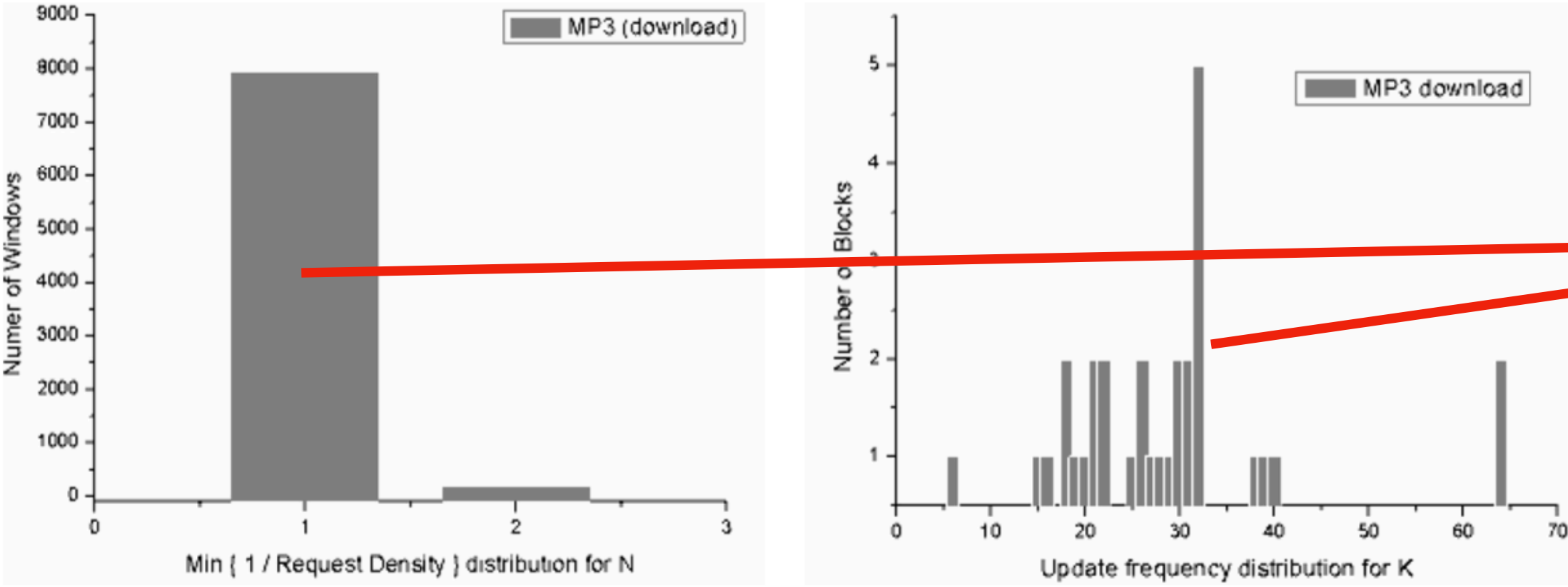
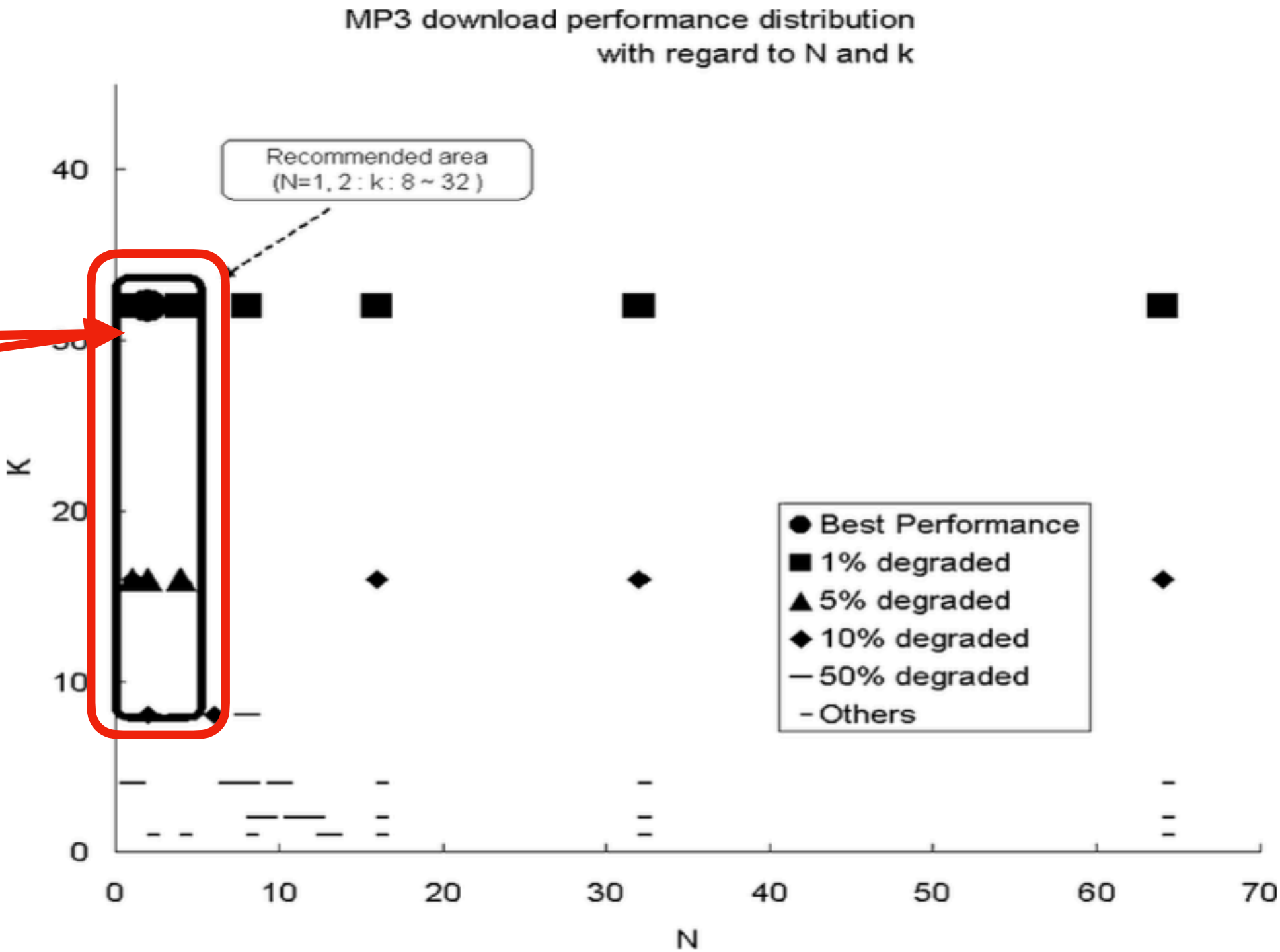


Fig. 17. Distribution of N and K values for the MP3 application.



MP3 download performance variation with the change of N and K .

Conclusion :

- FGM 아키텍처 소개.
- Request Density / 업데이트 빈도 기반 Workload Analysis 제시.
- 아키텍처의 유용성을 실험으로 증명.

Thank You!